Arquitectura de Solución
Sistema de Banca por Internet: BP

Documento de diseño de arquitectura (Modelo C4) · Ejercicio práctico de arquitectura de soluciones

Por Stanley Llaguno C

Tabla de contenido

1- Resumen Ejecutivo 4
2- Alcance y Supuestos 4
2.1 Alcance 4
2.2 Supuestos 4
3- Elementos normativos y de cumplimiento 5
4- Decisiones arquitectónicas clave 7
4.1 Delimitación de dominios (Bounded Contexts) 8
5- Diagrama de Contexto (C4, Nivel 1) 9
Actores y sistemas 9
6- Diagrama de Contenedores (C4, Nivel 2) 10
6.1 Canales, perímetro, BFF e identidad 10
6.2ª Servicios de dominio, integración y sistemas externos 11
6.2.b Datos, mensajería y auditoría 12
7- Diagrama de Componentes (C4, Nivel 3) 14
Patrones arquitectónicos aplicados 15
7.1 Segundo ejemplo, a menor escala: Servicio de Movimientos 17
8- Diagramas de secuencia complementarios 18
8.1 Autenticación SPA: OAuth2.0 Authorization Code + PKCE vía BFF (Token Handler) 18
8.2 Onboarding con reconocimiento facial 20
8.3 Transferencia (propia, intra-BP o interbancaria) con idempotencia, riesgo diferenciado y liquidación realista 21
9- Arquitectura de autenticación y autorización 23
9.1 Flujo OAuth2.0 recomendado 23
9.2 Dónde se almacena el token: BFF (SPA) vs. almacenamiento nativo (móvil) 23
9.3 Gestión de tokens 23
9.4 Método de acceso post onboarding 24
10- Arquitectura de Onboarding (reconocimiento facial) 24
10.1 Elección de herramienta de reconocimiento facial 24
10.2 Integración 25
11- Auditoría y persistencia de clientes frecuentes 25
11.1 Auditoría inmutable 25
11.2 Persistencia para clientes frecuentes 26
12- Capa de integración: API Gateway y servicios 28
12.1 Servicios base 28
12.2 Servicios adicionales propuestos 28
12.3 Gestión de contratos y versionado de APIs 29
13- Notificaciones 29
14- Alta disponibilidad, tolerancia a fallos, DR, seguridad, monitoreo y auto-healing 30

DevSecOps y seguridad de la cadena de suministro 31
Justificación de la estrategia HA/DR elegida 32
14.1 Diagrama de infraestructura (complementario) 33
15- Infraestructura Azure y consideraciones de costos 33
Palancas de optimización de costo 34
16- Arquitectura Front-End: SPA y Móvil 35
16.1 SPA Web 35
16.2 Aplicación móvil multiplataforma 36
17- Conclusiones 36

1- Resumen Ejecutivo

BP requiere modernizar su banca por internet permitiendo a sus clientes consultar el histórico de movimientos y realizar transferencias/pagos propios e interbancarios, desde una SPA web y una aplicación móvil multiplataforma.

La solución propuesta nos muestra una arquitectura desacoplada basada en microservicios, expuesta mediante un API Gateway, con autenticación estándar OAuth 2.0/OIDC y un proceso de onboarding digital con reconocimiento facial. Sumamos auditoría inmutable de toda acción del cliente y una capa de infraestructura sobre Microsoft Azure pensada para que la alta disponibilidad, tolerancia a fallos, recuperación ante desastres, seguridad y auto-healing.

El diseño se documenta bajo el modelo C4 (Contexto, Contenedores, Componentes), complementado con diagramas de secuencia para los siguientes flujos críticos:

- Autenticación
- Onboarding
- Transferencias

Y la justificación teórica de cada decisión arquitectónica relevante dado el caso de este ejercicio

2- Alcance y Supuestos

2.1 Alcance

- Consulta de datos básicos del cliente y de sus productos.
- Consulta de histórico de movimientos.
- Transferencias entre cuentas propias, a terceros dentro de BP e interbancarias.
- Onboarding digital de nuevos clientes en la app móvil con reconocimiento facial.
- Autenticación/autorización centralizada vía un proveedor OAuth2.0 ya existente en la compañía (no se implementa el servicio de identidad desde cero, se configura).
- Notificación al cliente de todo movimiento realizado, por al menos 2 canales.
- Auditoría de todas las acciones del cliente y mecanismo de persistencia optimizado para clientes frecuentes.

2.2 Supuestos

- BP ya opera una **plataforma Core bancaria** (cuentas, productos, movimientos) y un **sistema complementario de datos de cliente** (KYC extendido), ambos con APIs o colas propias que la capa de integración debe consumir.
- Existe presupuesto para infraestructura cloud administrada (PaaS): se prioriza reducir carga operativa (menos IaaS propio) sobre costo mínimo absoluto.
- El producto de identidad OAuth2.0 corporativo soporta OpenID Connect y los flujos de Authorization Code + PKCE y Client Credentials; solo se configura, no se construye.
- BP opera bajo regulación bancaria de un país con ley de protección de datos personales de tipo Habeas Data / RGPD. El diseño es agnóstico a la jurisdicción específica, pero cumple los principios comunes descritos en la sección 3.

3- Elementos normativos y de cumplimiento

Por tratarse de una entidad financiera, la arquitectura debe incorporar por diseño (*compliance by design*) los siguientes marcos normativos y de seguridad:

Ambito	Norma / estándar	Impacto en la arquitectura
Protección de datos personales	Ley de protección de datos personales / Habeas Data (según jurisdicción), principios equivalentes a GDPR	Cifrado en reposo y tránsito, minimización de datos expuestos entre servicios, consentimiento explícito en onboarding, derecho de acceso/rectificación/eliminación auditable.
Seguridad de datos de tarjetas	PCI DSS	Ningún dato sensible de tarjeta se almacena, procesa ni transmite por infraestructura de BP fuera de proveedores certificados; la tokenización ocurre en el proveedor, y BP solo maneja tokens no reversibles. Bajo este modelo, el alcance PCI de BP se reduce a SAQ-A, no SAQ-D, disminuyendo significativamente el esfuerzo y costo de auditoría anual. Se mantiene segmentación de red para los servicios que interactúan con el token La rotación de las llaves de tokenización es responsabilidad contractual del proveedor PCI DSS Level 1, con evidencia de rotación auditable trimestralmente como parte del ciclo de cumplimiento de BP.
Continuidad y riesgo operacional bancario	Lineamientos tipo Basilea / circulares del regulador financiero local sobre riesgo operativo y ciberseguridad	Exigencia de HA, DR con RTO/RPO definidos, planes de continuidad de negocio, monitoreo continuo (sección 14).
Prevención de lavado de activos	Normativa AML/KYC	El onboarding y las transferencias interbancarias deben dejar traza auditable completa y permitir reglas de scoring/alertas (ver servicio de riesgo, sección 12).
Estándar de autenticación	OAuth 2.0 / OpenID Connect (RFC 6749, RFC 6819, OIDC Core)	Flujo recomendado Authorization Code + PKCE (sección 9); tokens de vida corta, refresh tokens rotativos.
Cumplimiento del proveedor cloud	Certificaciones de Azure: ISO 27001, ISO 27017/27018, SOC 1/2/3, PCI DSS, y en general un catálogo de cumplimiento amplio orientado a sector financiero regulado	Justifica en parte la elección de Azure como nube (sección 4 y 15): reduce el esfuerzo de auditoría de infraestructura porque el proveedor ya está certificado sobre los controles que el regulador bancario suele exigir.

Estas normas no se implementan como "un módulo más": se reflejan de forma transversal en el diseño (cifrado, segmentación, auditoría inmutable, gestión de secretos, retención de datos) y se referencian en las secciones correspondientes.

Ejemplo ilustrativo (el diseño se mantiene agnóstico de jurisdicción): pensemos en un caso hipotético en Ecuador. Ahí aplicarían la Ley Orgánica de Protección de Datos Personales y la normativa de la Superintendencia de Bancos, que en esencia piden el mismo conjunto de controles que ya cubrimos arriba: cifrado,

trazabilidad auditable, continuidad de negocio con RTO/RPO definidos. Por eso la arquitectura no cambia según cuál jurisdicción aplique; solo cambia el detalle de reporte regulatorio que hay que implementar sobre la base ya diseñada.

Residencia de datos (data residency): los datos personales, financieros y de auditoría de los clientes de BP cabe recalcar que estos **no salen de la región Azure del país donde BP opera**: la región primaria y la región secundaria de DR (sección 14) se eligen dentro del mismo país o, si el país no tiene una segunda región Azure disponible, dentro de la región pareada más cercana que preserve la soberanía de datos exigida por el regulador local.

Esto aplica a todos los almacenes con datos de cliente (Azure SQL, Cosmos DB, Redis, el almacén de auditoría en sus dos niveles). Se garantiza a nivel de plataforma, restringiendo por Azure Policy qué regiones pueden usarse para desplegar cualquier recurso que persista datos de cliente.

Derecho al olvido vs. auditoría append-only: cuando un cliente ejerce su derecho de eliminación (Habeas Data/GDPR), el evento de auditoría no se borra, ya que el almacén es append-only por diseño (sección 11.1). En vez de eso se pseudonimiza: los identificadores personales del evento (nombre, documento, correo) se reemplazan por un hash irreversible, generado con una clave de pseudonimización propia del cliente que también se destruye en el proceso. El evento permanece como evidencia de que la acción ocurrió, un requisito regulatorio de continuidad y AML, pero deja de ser atribuible a una persona identificable. Este mecanismo se aplica sobre el nivel operativo (Azure SQL append-only); el archivado en Data Lake conserva la misma pseudonimización al momento del archivado periódico.

Clasificación de datos: no todo lo que maneja el sistema tiene el mismo nivel de sensibilidad ni la misma política de acceso. Adoptamos cuatro niveles: Restringido (datos biométricos del onboarding, tokens de pago, claves criptográficas: acceso solo por el servicio propietario, nunca en logs), Confidencial (saldos, movimientos, datos KYC: acceso por rol autenticado, cifrado en tránsito y reposo), Interno (métricas operativas, configuración de servicios: acceso por equipo técnico de BP) y Público (documentación de API en el portal de desarrolladores, catálogo de productos). Esta clasificación determina qué campos se excluyen de la telemetría (sección 14) y qué almacenes requieren Customer Managed Keys.

4- Decisiones arquitectónicas clave

Toda decisión relevante se documenta con al menos dos justificaciones teóricas y las alternativas evaluadas, según lo exigido por el ejercicio.

Decisión	Opciones evaluadas	Elegida	Justificación (≥2)
Estilo arquitectónico backend	Monolito modular · Microservicios · Serverless puro	Microservicios desplegados en contenedores (AKS)	**Principio de responsabilidad única a nivel de servicio**: cada dominio (datos básicos, movimientos, transferencias, riesgo) cambia y escala por razones distintas; un monolito acopla ciclos de despliegue y aumenta el radio de impacto de un fallo, algo que en banca no se puede permitir. **Escalabilidad diferenciada**: transferencias e histórico de movimientos tienen perfiles de carga muy distintos (picos de escritura vs. lectura intensiva); microservicios permiten escalar cada uno de forma independiente, mientras que serverless puro se descarta por latencia de cold-start y menor control sobre conexiones persistentes a la plataforma Core, un punto que en este dominio pesa mucho.
Capa de integración	Comunicación directa cliente→backend · ESB tradicional · API Gateway + microservicios	API Gateway (Azure API Management)	**Patrón Backend-for-Frontend / Gateway centralizado**: concentra autenticación, rate-limiting, versionado y observabilidad en un único punto, evitando que cada cliente (SPA, móvil) reimplemente esas políticas. Comparado con un ESB tradicional, un API Gateway es más liviano y orientado a REST/HTTP, con menor acoplamiento y menor costo operativo; el ESB se descarta porque introduce lógica de orquestación pesada donde solo se necesita enrutamiento, seguridad perimetral y políticas.
Proveedor cloud	AWS · Azure · GCP · Infraestructura on premise	Microsoft Azure	**Alineación regulatoria**: Azure cuenta con un catálogo de cumplimiento (ISO 27001, SOC 2, PCI DSS, Azure Compliance Manager) especialmente fuerte en el sector financiero regulado, y ofrece regiones con residencia de datos configurable, relevante para las leyes de datos personales. **Integración nativa de identidad**: Azure AD B2C/B2B se integra de forma directa con el resto del ecosistema (API Management, Key Vault, Monitor), reduciendo el número de piezas a integrar para el flujo OAuth2.0 exigido, frente a ensamblar un stack equivalente en AWS o GCP.
Comunicación entre microservicios	Llamadas síncronas REST únicamente - Mensajería asíncrona únicamente ·Híbrido (REST síncrono + eventos asíncronos)	Híbrido: REST/HTTPS síncrono para consultas y orquestación de transferencia; eventos asíncronos (Azure Service Bus)	Las consultas (datos básicos, movimientos) requieren respuesta inmediata al usuario, por lo que un patrón síncrono request/response es el adecuado; forzar eventos asíncronos ahí solo añadiría latencia percibida sin beneficio. La auditoría y las notificaciones son *fire-and-forget* desde la perspectiva del flujo principal: acoplarlas de forma síncrona haría que un fallo del servicio de notificaciones pudiera bloquear una transferencia. El patrón **Publish/Subscribe** desacopla temporalmente productor y consumidores y habilita reintentos sin afectar la transacción principal.

		para auditoría y notificaciones	

4.1 Delimitación de dominios (Bounded Contexts)

La segmentación en microservicios no sigue un criterio técnico como por ejemplo "un servicio por tabla", sino un ejercicio de Domain-Driven Design sobre el negocio bancario: cada microservicio corresponde a un *bounded context* con su propio lenguaje ubicuo, ciclo de vida y reglas de negocio.

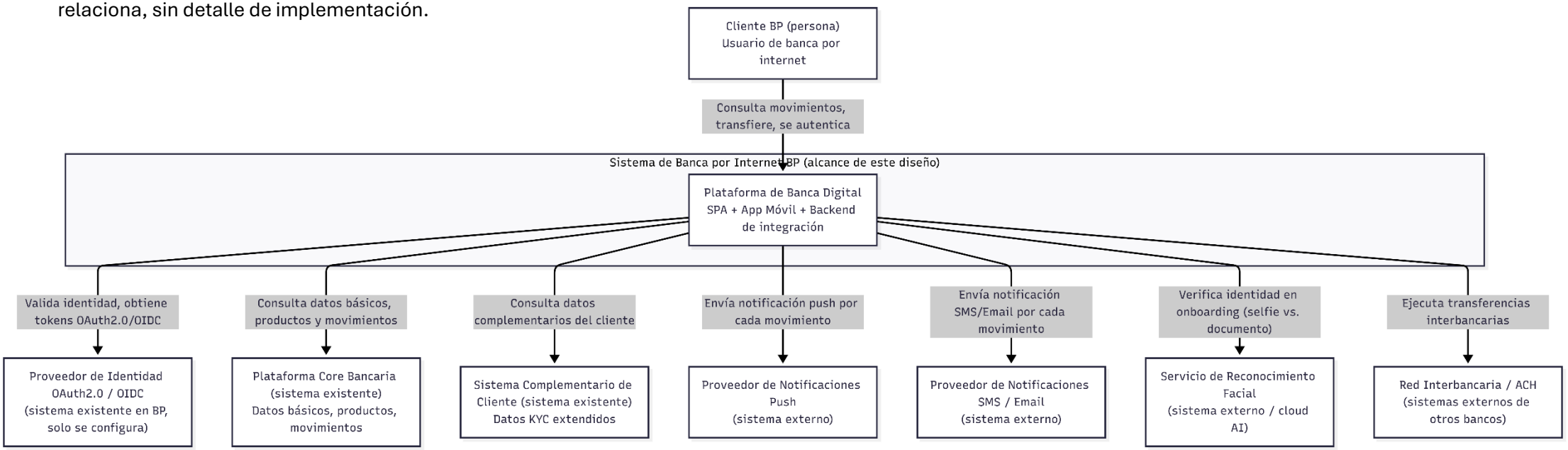
Los contextos identificados son: Datos Básicos de Cliente (perfil, productos), Movimientos (histórico transaccional, solo lectura) y Transferencias (operaciones que mutan saldo, con orquestación multipaso). También Riesgo/Antifraude (scoring, reglas iterativas), Onboarding (alta de nuevos clientes, integración con IDV), Auditoría (registro inmutable) y Notificaciones (fan out de eventos hacia canales).

Elegimos estos límites por dos razones. Primero, cada contexto cambia por razones distintas y a ritmos distintos: las reglas de riesgo se ajustan semanalmente por el área de fraude, las reglas de transferencia cambian por regulación, y el histórico de movimientos apenas cambia estructuralmente. Unirlos forzaría a un ciclo de release compartido que no refleja la realidad del negocio. Segundo, el vocabulario del negocio es distinto en cada contexto: "cliente" para Datos Básicos es un titular con productos, para Riesgo es un sujeto de scoring con historial de comportamiento, y para Onboarding es un prospecto sin identidad aún emitida. Forzar un modelo compartido de "cliente" acoplaría semánticas incompatibles.

El *context mapping* entre estos dominios se resuelve vía eventos de dominio publicados a Service Bus, evitando integración punto a punto entre servicios y respetando la autonomía de cada contexto sobre su modelo interno.

5- Diagrama de Contexto (C4, Nivel 1)

Audiencia: no técnica. Muestra el sistema de Banca por Internet BP como una caja única, sus usuarios y con qué sistemas externos e internos existentes se relaciona, sin detalle de implementación.



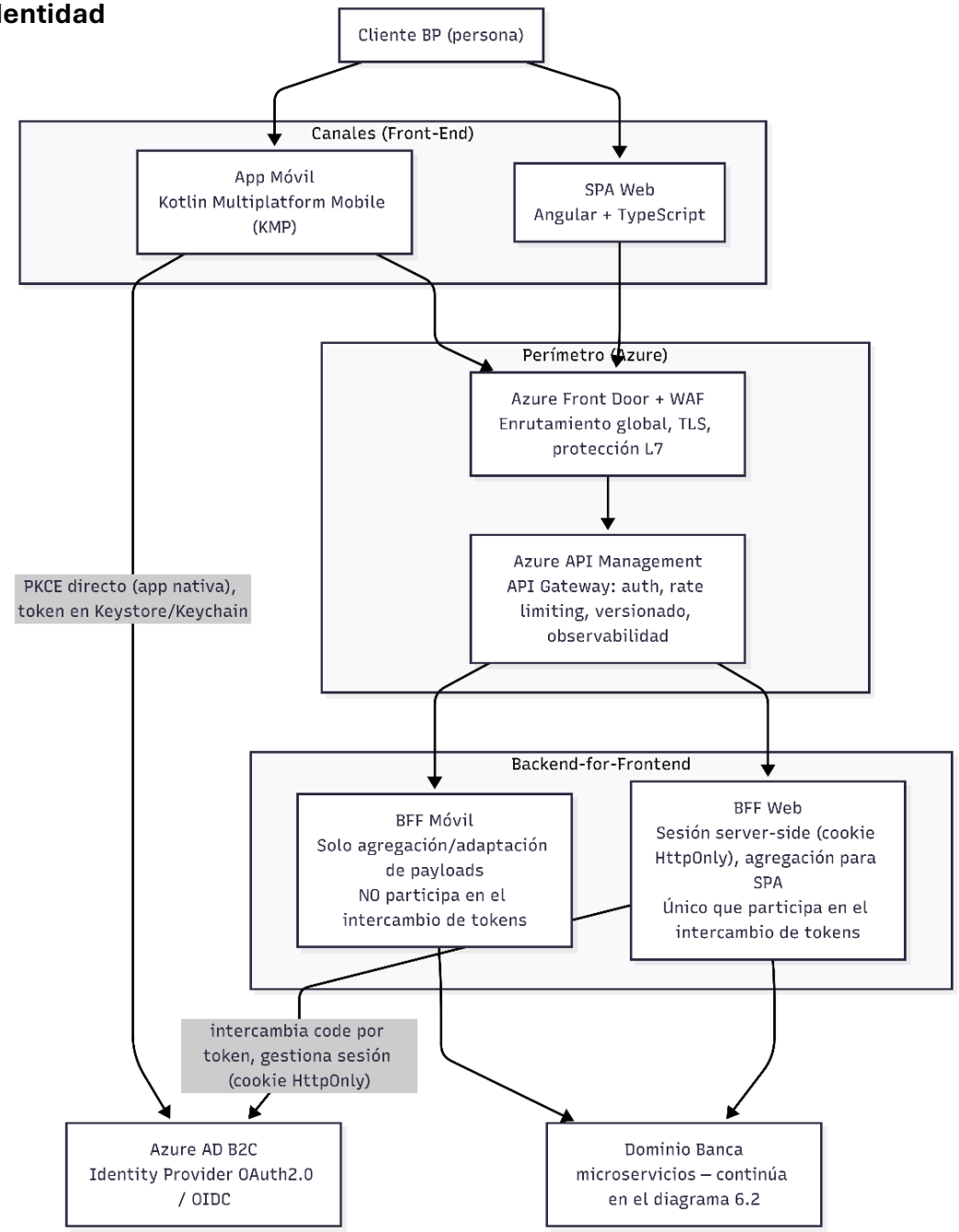
Actores y sistemas

Elemento	Tipo	Descripción
Cliente BP	Persona	Usuario final que consulta movimientos y realiza pagos/transferencias desde web o móvil.
Sistema de Banca por Internet BP	Sistema (en alcance)	La solución diseñada en este documento: canales SPA/móvil + backend de integración.
Proveedor de Identidad OAuth2.0/OIDC	Sistema existente	Producto corporativo de identidad ya disponible; se configura, no se construye.
Plataforma Core Bancaria	Sistema existente	Fuente de verdad de cuentas, productos y movimientos.
Sistema Complementario de Cliente	Sistema existente	Datos de cliente en detalle (KYC extendido) cuando el Core no basta.
Proveedores de notificación (2)	Sistema externo	Canales push y SMS/Email para cumplir la exigencia normativa de notificar cada movimiento.
Servicio de reconocimiento facial	Sistema externo	Verifica identidad del nuevo cliente durante el onboarding móvil.
Red interbancaria / ACH	Sistema externo	Procesan las transferencias hacia cuentas de otros bancos.

6- Diagrama de Contenedores (C4, Nivel 2)

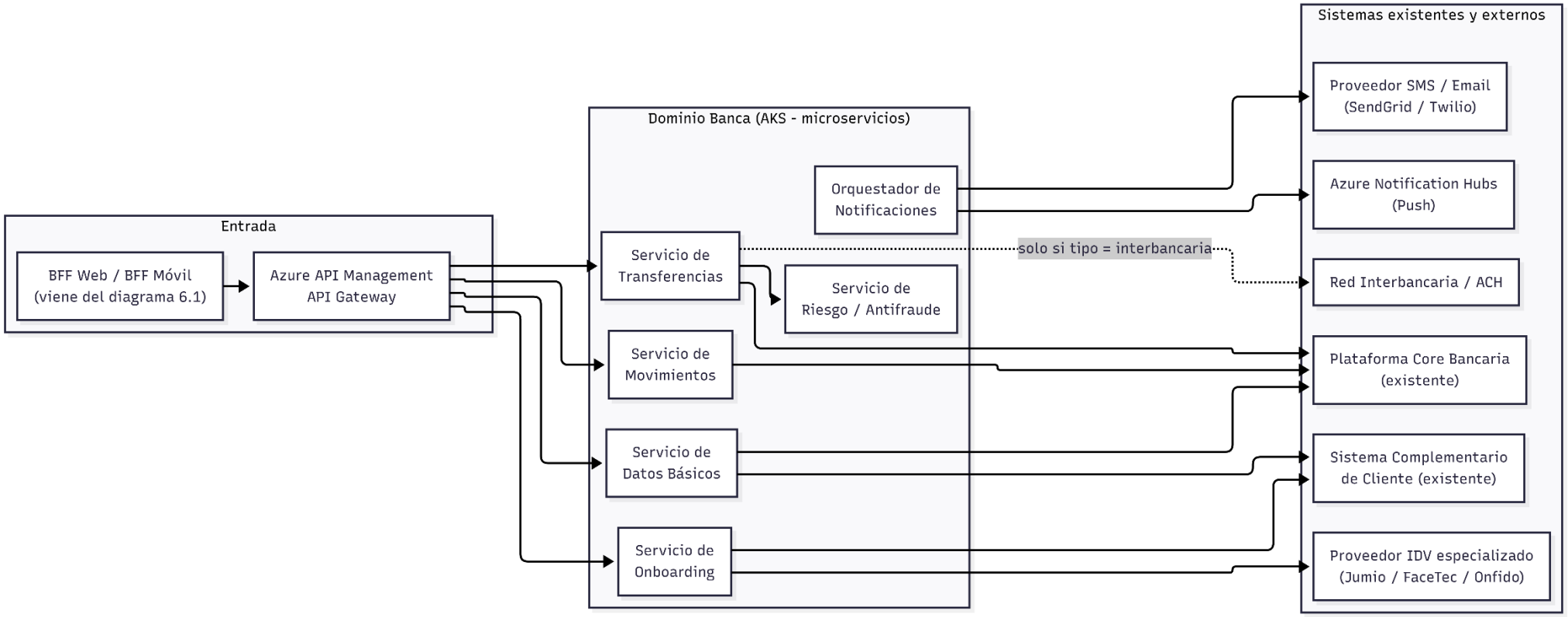
Audiencia: técnica. Abre la caja "Sistema de Banca por Internet BP" en sus aplicaciones, servicios, bases de datos y mensajería, incluyendo los componentes cloud (Azure) usados, sin bajar al detalle interno de cada microservicio.

6.1 Canales, perímetro, BFF e identidad



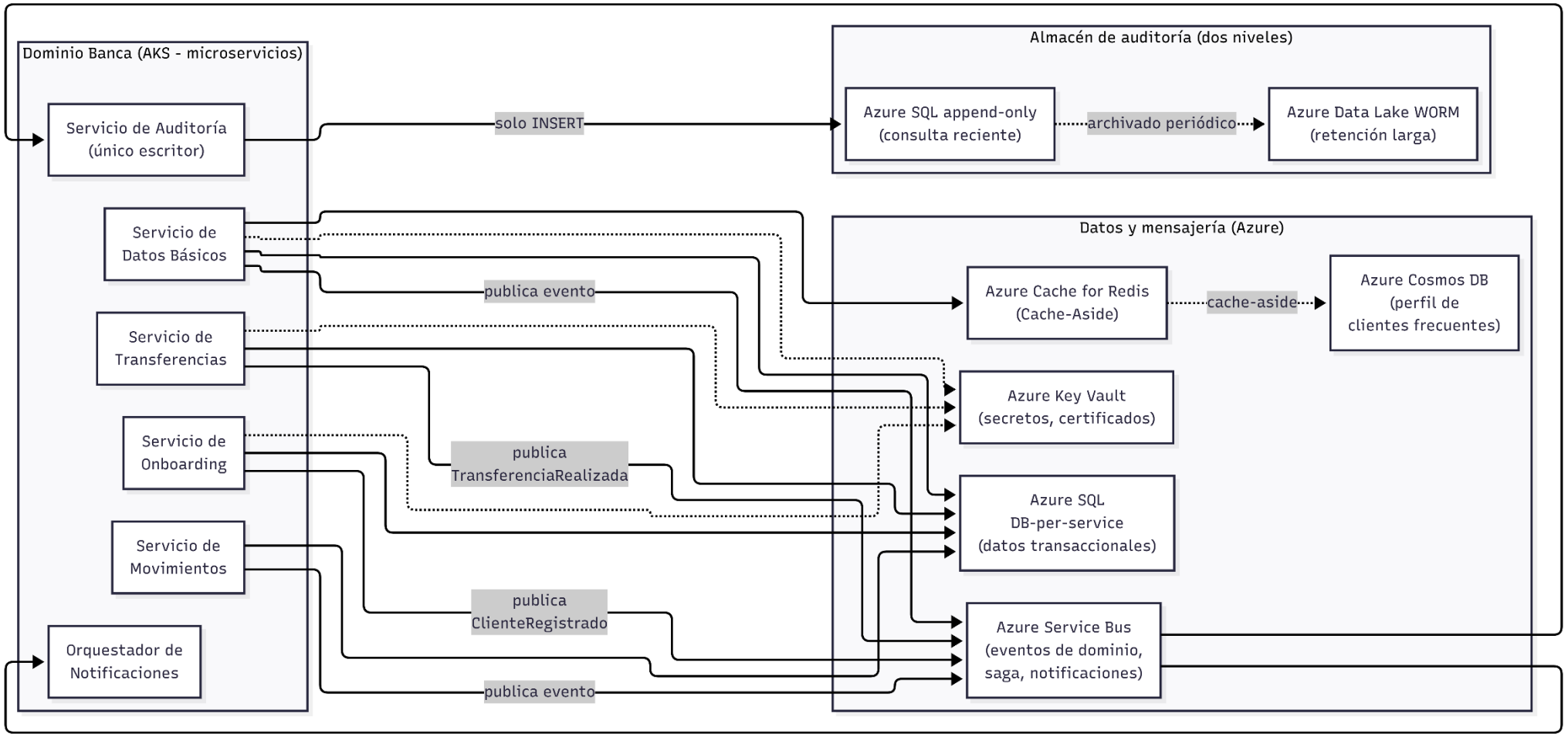
6.2ª Servicios de dominio, integración y sistemas externos

Cómo entra el tráfico y con qué sistemas de afuera habla cada servicio (flujo síncrono).



6.2.b Datos, mensajería y auditoría

Dónde se guarda cada cosa y cómo fluyen los eventos (persistencia + mensajería asíncrona).



Notas sobre contenedores clave

- **Azure Front Door + WAF:** punto único de entrada global, con protección contra ataques L7 (OWASP core rule set) y enrutamiento a la región activa (soporta el DR de la sección 14).
- **Azure API Management:** implementa el patrón API Gateway; centraliza validación de tokens OAuth2.0, throttling y contratos de API versionados para SPA y móvil.

¿Por qué Front Door y API Management, si ambos hacen TLS/WAF?

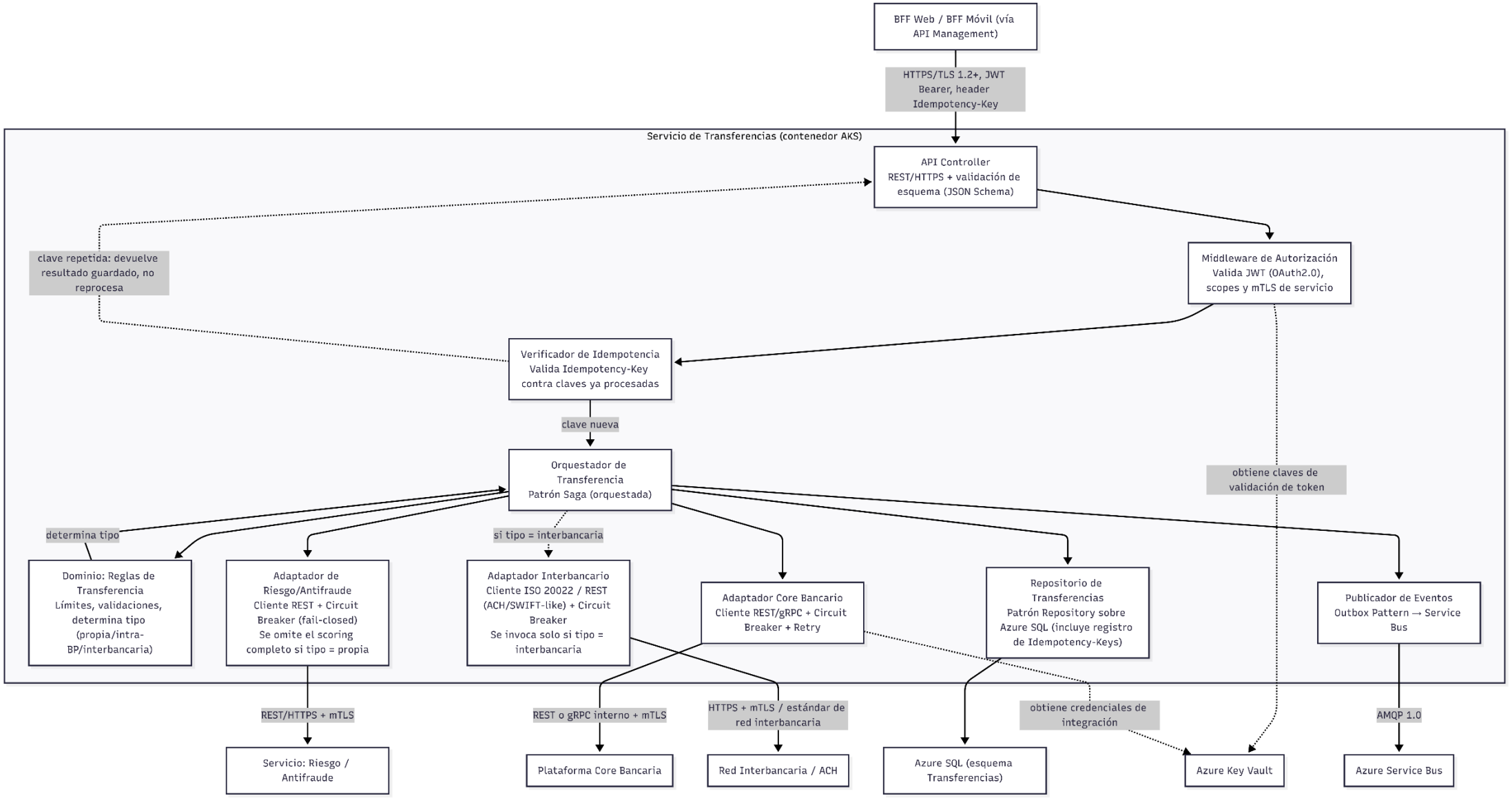
No son redundantes: Front Door opera en el borde global (anycast, enrutamiento entre regiones, failover de región activa) y es agnóstico al contenido de la API; API Management opera a nivel de contrato (versionado, cuotas por API, transformación de payload, portal de desarrolladores). Uno resuelve a qué región llega la petición, el otro qué política aplica a esa API en particular. Eliminar cualquiera de los dos obligaría al otro a asumir responsabilidades para las que no fue diseñado.

BFF Web / BFF Móvil: cada canal tiene su propio Backend for Frontend (justificado en la sección 12.2) que agrega y adapta las respuestas de los microservicios de dominio a la forma que cada front necesita. El BFF Web además concentra la sesión del usuario server side (detalle en sección 9.1), evitando exponer tokens al navegador.

- **Base de datos por servicio (Database perService):** cada microservicio del dominio bancario es dueño de su propio esquema/base, evitando acoplamiento a nivel de datos entre servicios que cambian a ritmos distintos.
- **Auditoría en dos niveles:** Azure SQL append-only para consulta operativa reciente + archivado periódico a Azure Data Lake en modo WORM para retención regulatoria de largo plazo (detalle en sección 11.1).
- **Azure Service Bus:** columna vertebral de comunicación asíncrona (tópicos/subscripciones) para auditoría, notificaciones y el patrón Saga de transferencias (ver diagrama de componentes de la sección 7 y el diagrama de secuencia 8.3).

7- Diagrama de Componentes (C4, Nivel 3)

Mayor detalle técnico. Se profundiza en el contenedor más complejo del dominio, el Servicio de Transferencias, mostrando sus componentes internos, patrones arquitectónicos aplicados y protocolos de comunicación con seguridad. Los servicios de Datos Básicos y Movimientos siguen una estructura interna análoga (capa API → aplicación → dominio → adaptadores), simplificada por no requerir orquestación multipaso.



Patrones arquitectónicos aplicados

Patrón	Dónde se usa	Justificación (≥2)
Saga (orquestada)	Orquestador de Transferencia	Una transferencia interbancaria toca múltiples sistemas (riesgo, Core, red externa) sin una transacción distribuida ACID posible entre ellos; Saga coordina pasos con compensación (ej. reversar débito si el banco destino rechaza) en vez de bloquear recursos entre servicios. Frente a Saga coreografiada (basada solo en eventos), la variante orquestada da trazabilidad centralizada del estado de la transacción, algo que la auditoría bancaria necesita para poder responder "en qué paso quedó esta transferencia" sin reconstruir el flujo a partir de eventos dispersos.
Circuit Breaker + Retry	Adaptadores de Riesgo, Core y Red Interbancaria	Aísla fallos de sistemas externos (Core, banco destino) para que no se propaguen como cascada de timeouts dentro del servicio de transferencias (principio de *bulkhead/failfast*). Soporta directamente el requisito de tolerancia a fallos: ante caída parcial de un sistema externo, el circuito abre y el servicio responde de forma degradada/controlada en vez de agotar hilos o conexiones. **Política declarada por adaptador** (no es lo mismo "degradado" para todos): el circuito del **Adaptador de Riesgo** es **fail-closed:** si no hay score disponible, no se autoriza la transferencia (excepto tipo propia), porque dejar pasar sin evaluar fraude es un riesgo mayor que rechazar temporalmente. El circuito del **Adaptador Core** no tiene modo degradado seguro: sin poder confirmar saldo/débito no hay forma correcta de continuar, así que "degradado" ahí significa simplemente fallar rápido y limpio, no un intento de aproximar la operación.
Outbox Pattern	Publicador de Eventos	Garantiza consistencia entre la escritura transaccional en la base de datos del servicio y la publicación del evento a Service Bus (ambas ocurren o ninguna), evitando el problema clásico de "guardé pero no notifiqué" o viceversa. Evita necesitar un patrón de transacción distribuida (2PC) entre base de datos y broker de mensajería, que es costoso y frágil operativamente.
Repository	Acceso a datos en cada microservicio	Desacopla la lógica de dominio del motor de persistencia concreto, facilitando pruebas unitarias del dominio sin base de datos real. Punto único de cambio si se migra de Azure SQL a otro motor, sin tocar reglas de negocio.

Idempotency Key	Verificador de Idempotencia (entrada al Orquestador de Transferencia)	En una API de pagos, un timeout de red no le dice al cliente si la transferencia se ejecutó o no; sin una clave de idempotencia generada por el cliente y verificada antes de procesar, un reintento automático (del app o de un proxy intermedio) puede ejecutar la misma transferencia dos veces, y ese es un error que un banco no se puede permitir. Resolverlo a nivel de plataforma (verificación temprana, antes del Orquestador) evita que cada nuevo tipo de transferencia tenga que reimplementar su propia protección contra duplicados, y permite responder de forma inmediata con el resultado ya calculado en vez de volver a tocar el Core o la red interbancaria. Las claves se retienen 24 a 48 horas antes de purgarse, ventana suficiente para cubrir reintentos legítimos de red y de clientes móviles que reabren la app con conexión intermitente, sin que el índice de idempotencia crezca de forma indefinida.

Seguridad en las comunicaciones

- **Cliente → Gateway → BFF:** el navegador de la SPA nunca ve el JWT (sección 9.1), le llega al BFF vía cookie de sesión HttpOnly. La app móvil es distinta: retiene el token en su almacenamiento seguro nativo (Keystore/Keychain) y lo manda como Bearer. En los dos casos, eso sí, HTTPS/TLS 1.2+ y validación en APIM antes de reenviar la petición.

- **BFF/Gateway → microservicios y entre microservicios:** mTLS dentro del clúster AKS vía service mesh (Istio, como add-on gestionado de AKS), más validación de scopes por endpoint. Es zero trust interno puro: no asumimos confianza solo por estar dentro de la red. Elegimos Istio sobre alternativas como Linkerd porque su ecosistema de políticas de autorización L7 está más maduro y porque viene como add-on gestionado por la plataforma, lo que nos saca de encima instalar y mantener el mesh a mano.

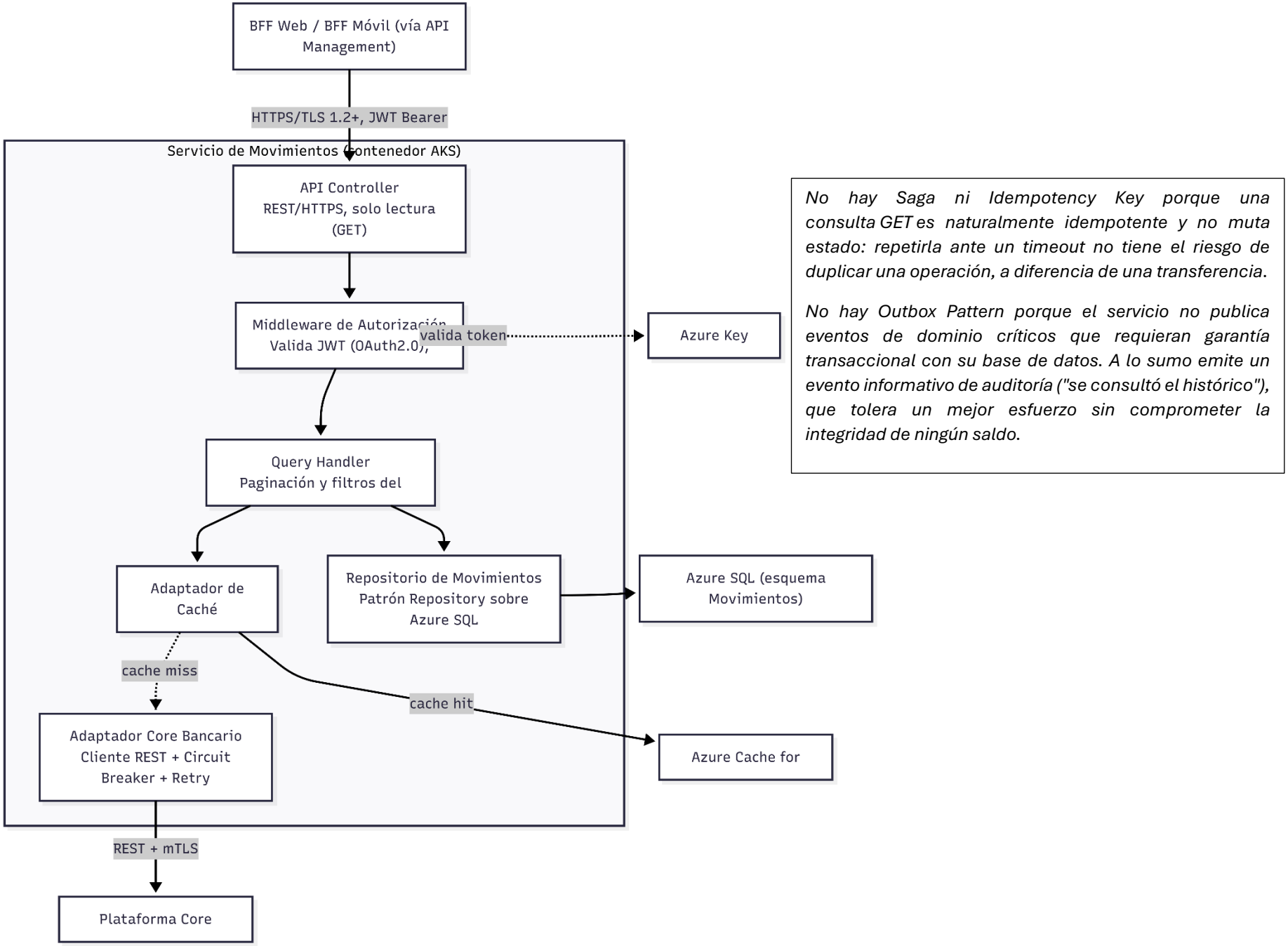
- **Microservicio → sistemas externos (Core, banco, riesgo):** mTLS o TLS + credenciales gestionadas por Azure Key Vault, nunca en configuración estática.

- **Microservicio → Service Bus:** AMQP 1.0 sobre TLS, autenticación con Managed Identity (sin secretos embebidos en el servicio).

El contraste entre Movimientos (solo lectura, sin mutación de estado) y Transferencias (escritura orquestada con compensación) aplica el principio de CQRS (Command Query Responsibility Segregation), que separa el modelo de consulta del modelo de mando para poder optimizar cada uno por su cuenta. Movimientos se beneficia de caché agresiva y modelos de lectura desnormalizados sin que eso comprometa la consistencia transaccional. Transferencias, en cambio, prioriza integridad, idempotencia y trazabilidad de escritura, sin que las decisiones de rendimiento del lado de consulta la condicionen.

7.1 Segundo ejemplo, a menor escala: Servicio de Movimientos

Para no apoyar la evaluación de profundidad en un único caso, detallamos también el servicio de consulta más simple. Sirve como contraste directo: mismo estilo de capas (Controller → Autorización → lógica de aplicación → adaptadores → Repository), pero sin Saga, es decir sin necesidad de compensación entre pasos, sin Idempotency Key y sin Outbox. La razón es que son innecesarios aquí, no que se hayan omitido por descuido.



No hay Saga ni Idempotency Key porque una consulta GET es naturalmente idempotente y no muta estado: repetirla ante un timeout no tiene el riesgo de duplicar una operación, a diferencia de una transferencia.

No hay Outbox Pattern porque el servicio no publica eventos de dominio críticos que requieran garantía transaccional con su base de datos. A lo sumo emite un evento informativo de auditoría ("se consultó el histórico"), que tolera un mejor esfuerzo sin comprometer la integridad de ningún saldo.

8- Diagramas de secuencia complementarios

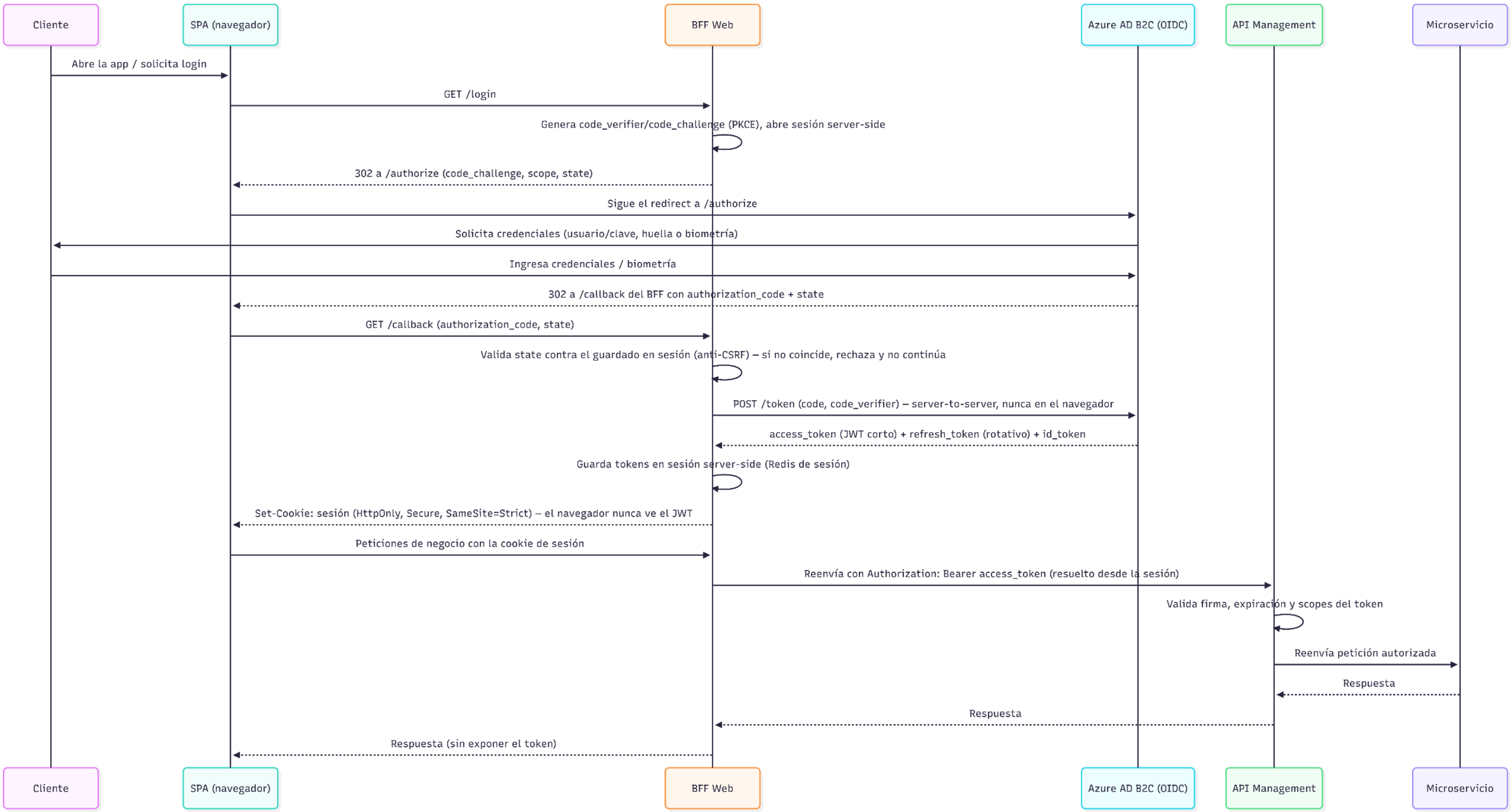
Estos diagramas no son exigidos explícitamente como C4, pero son necesarios y recomendados para dar claridad a los 3 flujos de calificación que se evalúan de forma puntual según reglamentos y buenas prácticas: autenticación, onboarding y el recorrido de una transferencia.

8.1 Autenticación SPA: OAuth2.0 Authorization Code + PKCE vía BFF (Token Handler)

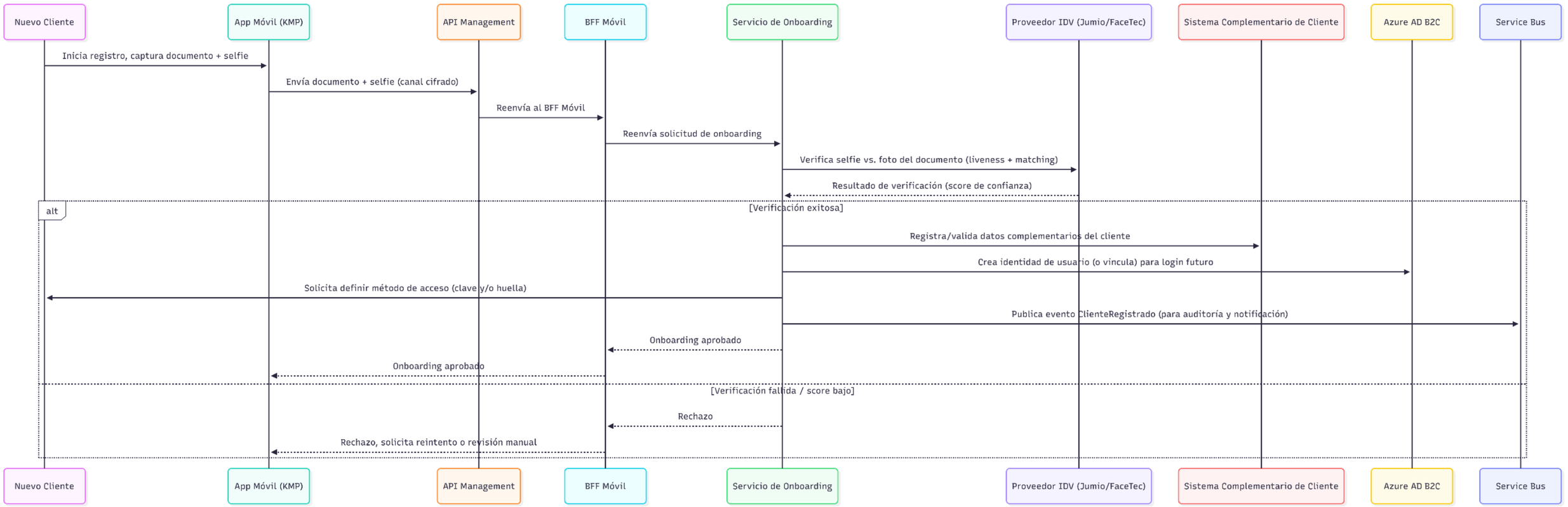
La app móvil sigue el flujo PKCE directo que describimos conceptualmente en la sección 9.1: el token queda en el Keystore/Keychain del dispositivo, un almacenamiento seguro que el navegador simplemente no tiene. La SPA es distinta, usa el BFF Web como intermediario, así el navegador nunca recibe el JWT, solo una cookie de sesión.

Es la única forma de que el BFF propuesto en la sección 12.2 cumpla su función donde más importa. Si el token igual termina en el navegador, la reducción de superficie de ataque frente a robo vía XSS queda solo en el papel.

Autenticación SPA: OAuth2.0 Authorization Code + PKCE vía BFF (Token Handler)



8.2 Onboarding con reconocimiento facial

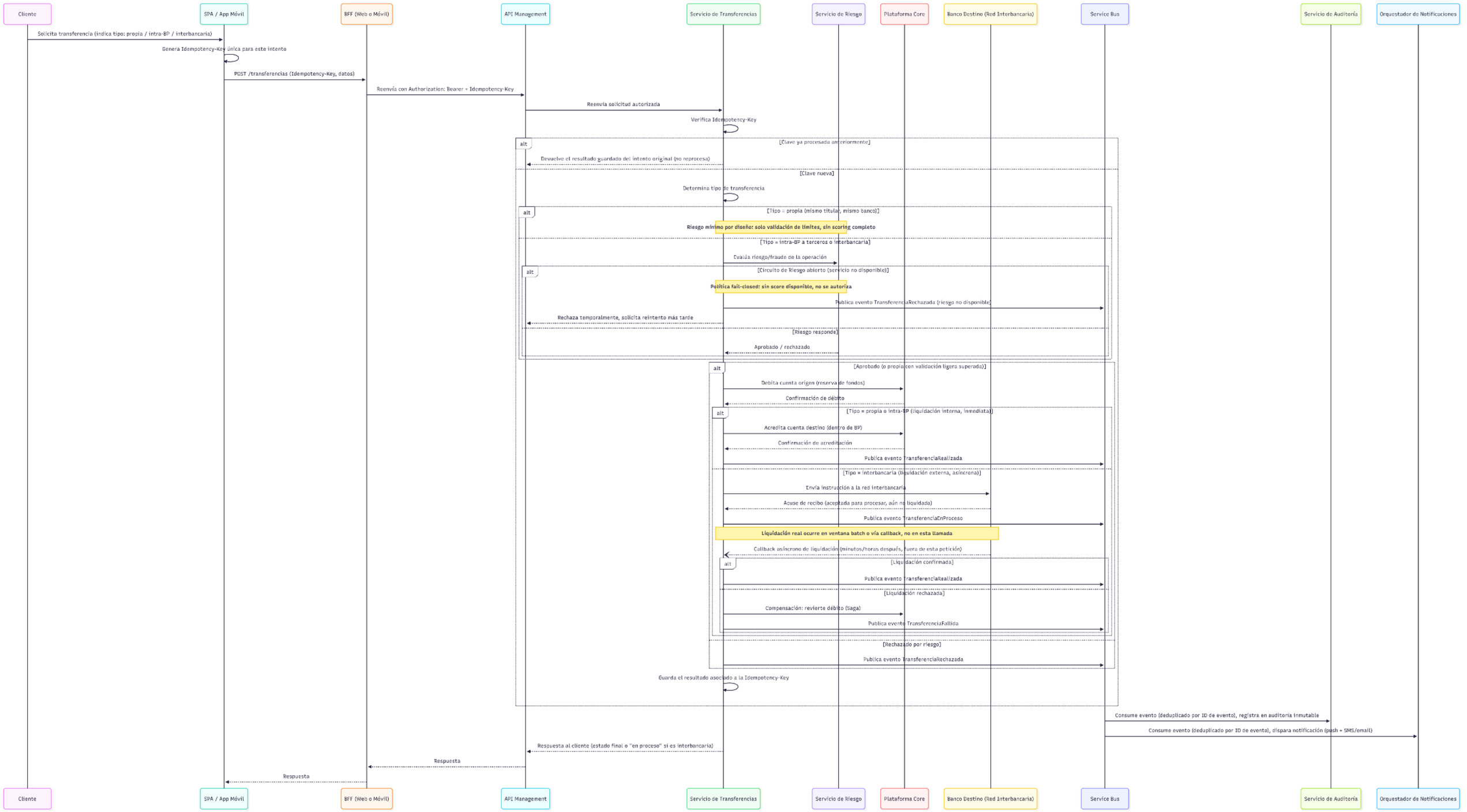


8.3 Transferencia (propia, intra-BP o interbancaria) con idempotencia, riesgo diferenciado y liquidación realista

El flujo cubre cinco aspectos que una transferencia bancaria real no puede dejar de lado. Protección contra duplicados vía IdempotencyKey. Diferenciación del llamado a servicios externos según el tipo de transferencia. Validación de riesgo proporcional al tipo: las transferencias **propias** (mismo titular, mismo banco) solo llevan validación ligera de límites, no scoring completo. Una política fail-closed explícita si el Servicio de Riesgo no está disponible. Y un modelo de liquidación interbancaria fiel a cómo funciona en la práctica**: asíncrona**, con acuse de recibo inmediato y confirmación real vía callback o lote posterior, nunca dentro de la misma llamada.

Una transferencia interbancaria puede quedar en estado EN_PROCESO visible para el cliente (con su propia notificación) y resolverse a REALIZADA o FALLIDA más tarde vía el callback asíncrono. El cliente no se queda "esperando" la respuesta HTTP hasta que el banco destino liquide.

Transferencia (propia, intra-BP o interbancaria) con idempotencia, riesgo diferenciado y liquidación realista



9- Arquitectura de autenticación y autorización

9.1 Flujo OAuth2.0 recomendado

Se recomienda Authorization Code Flow con PKCE (Proof Key for Code Exchange) para ambos canales (SPA y app móvil), configurado sobre el proveedor de identidad corporativo (Azure AD B2C u equivalente OIDC).

SPA y app móvil son clientes públicos: no pueden custodiar un client secret de forma segura. El flujo Implicit, histórico para SPA, expone el token directamente en el fragmento de la URL y no soporta refresh tokens de forma segura. PKCE resuelve exactamente el riesgo de interceptación del código de autorización sin necesitar secreto de cliente. Hoy es la recomendación explícita de la RFC 8252 y de las mejores prácticas de OAuth2.0 (BCP 212) para apps nativas y SPA.

Permite desacoplar completamente la lógica de autenticación del backend de BP. Los microservicios solo validan el JWT emitido por el IdP (firma, expiración, scopes), sin conocer credenciales ni gestionar sesiones. Así se cumple el requisito de "no implementar la lógica, solo configurar el producto existente".

Descartamos Client Credentials para tráfico de usuario final, es para comunicación servicio a servicio, por ejemplo jobs internos. También descartamos Resource Owner Password Credentials, porque expone las credenciales del usuario directamente a la app cliente y eso contradice el principio de menor privilegio de OAuth 2.0.

9.2 Dónde se almacena el token: BFF (SPA) vs. almacenamiento nativo (móvil)

PKCE resuelve la interceptación del código de autorización, pero no resuelve dónde queda el token después. Y ahí SPA y móvil se tratan distinto (ver diagrama de secuencia 8.1):

La app móvil retiene el token directamente, porque el sistema operativo le da almacenamiento respaldado por hardware (Android Keystore / iOS Keychain, ligado al Secure Enclave) que un proceso de otra app no puede leer. Así lo recomienda la RFC 8252 para apps nativas. Agregar un BFF ahí no reduciría el riesgo, solo sumaría latencia.

La SPA corre en el navegador, que no tiene nada parecido a ese almacenamiento seguro. Cualquier token guardado en localStorage, sessionStorage o hasta en una variable en memoria queda al alcance de un ataque XSS exitoso. Por eso el BFF Web (sección 12.2) hace el intercambio PKCE por el navegador y solo entrega una cookie HttpOnly/Secure/SameSite=Strict. Sigue el patrón Token Handler que recomienda el borrador IETF OAuth 2.0 for Browser-Based Apps justo para este escenario.

9.3 Gestión de tokens

- **access_token**: JWT de vida corta (5-15 min), validado por API Management en cada petición (firma + expiración + scopes/claims).
- **refresh_token:** es rotativo, se invalida el anterior al usar uno nuevo (un patrón conocido como refresh token rotation). En móvil se almacena en Keychain/Keystore. En el caso de la SPA, vive únicamente en la sesión server side del BFF Web, nunca en el navegador.

- **Step up authentication:** para operaciones de alto riesgo, como transferencias sobre cierto monto o a nuevos beneficiarios, exigimos una reautenticación o confirmación biométrica adicional antes de autorizar. No basta la presencia de un token válido.

9.4 Método de acceso post onboarding

Después del onboarding inicial (autenticado con reconocimiento facial, sección 10), el cliente elige su método de acceso recurrente. Recomendamos biometría vía FIDO2/WebAuthn en la SPA y biometría nativa (huella dactilar / Face ID) en móvil, con usuario y clave como respaldo.

FIDO2/WebAuthn y la biometría nativa móvil usan criptografía de clave pública, con la clave privada anclada al secure enclave/TPM del dispositivo. La credencial nunca viaja ni se almacena en el servidor. Eso elimina el riesgo de robo masivo de credenciales por brecha de base de datos, a diferencia de usuario/clave.

Reduce fricción y mejora la tasa de éxito de login frente a clave tradicional, sin sacrificar seguridad. El segundo factor (posesión del dispositivo + biometría) es inherente al método, así que cumple con autenticación multifactor sin pasos adicionales visibles para el usuario.

10- Arquitectura de Onboarding (reconocimiento facial)

El onboarding se modela como parte del flujo de autenticación y autorización, no como un proceso aparte. Solo tras verificar exitosamente la identidad del nuevo cliente se crea su identidad en el proveedor OAuth2.0, y ahí queda habilitado para autenticarse (ver diagrama de secuencia 8.2).

10.1 Elección de herramienta de reconocimiento facial

Se evaluaron dos categorías de solución:

Opción	Descripción	Consideración
Azure AI Vision Face API	Servicio genérico de detección/verificación facial de Azure	Encaja de forma nativa con el resto del stack, pero Microsoft restringe el acceso a las capacidades de identificación/verificación facial bajo su política de *Responsible AI* (requiere aprobación de acceso limitado) y no está certificado específicamente para detección de ataques de presentación (spoofing) en procesos KYC.
Proveedor especializado de verificación de identidad (IDV) (FaceTec, Jumio u Onfido, por ejemplo)	Plataforma dedicada a KYC digital: liveness detection + comparación selfie-documento + validación de autenticidad del documento	Esta es la elegida justificación abajo.

Justificacion

Los proveedores especializados certifican su liveness detection contra el estándar ISO/IEC 30107-3 (Presentation Attack Detection). Es decir, están probados formalmente contra ataques con foto impresa, video o máscara. Es un requisito para que el onboarding sea defendible ante el regulador como evidencia válida de KYC, algo que un API de propósito general no garantiza.

Estas plataformas ya integran la comparación selfie vs documento y la validación de autenticidad del documento de identidad como hologramas, MRZ, etc. En un solo flujo. Eso reduce la superficie de integración frente a ensamblar detección de vida, verificación de documento y matching facial con servicios separados.

10.2 Integración

El **Servicio de Onboarding** orquesta la llamada al proveedor IDV (vía su SDK móvil embebido en la app KMP + API REST de verificación en backend), y solo si el resultado es exitoso:

- Registra/valida los datos complementarios en el Sistema Complementario de Cliente (KYC extendido).
- Crea la identidad del usuario en Azure AD B2C mediante un custom policy (Identity Experience Framework). Esa policy invoca un REST API technical profile hacia el Servicio de Onboarding, para confirmar la verificación antes de emitir la cuenta.
- Publica el evento ClienteRegistrado a Service Bus para auditoría y notificación de bienvenida.

Esto asegura que el reconocimiento facial es parte del flujo de autorización (no se emite identidad sin verificación exitosa) y no un simple paso de UI desconectado del backend de identidad.

11- Auditoría y persistencia de clientes frecuentes

11.1 Auditoría inmutable

Se aplica un patrón de log de auditoría basado en eventos (event-driven audit log). Cada microservicio del dominio publica un evento de dominio por cada acción relevante del cliente (login, consulta, transferencia, cambio de datos) a Azure Service Bus. El Servicio de Auditoría es el único consumidor autorizado a escribir en el almacén de auditoría, en modo append-only.

Azure Service Bus entrega mensajes con garantía at-least-once: el mismo evento puede llegar duplicado. Cada evento se publica con un EventId único (GUID) y la deduplicación se resuelve en dos capas complementarias. Primero, la duplicate detection nativa de Service Bus a nivel de tópico, que descarta reentregas dentro de una ventana configurable de hasta 10 minutos, cubriendo el escenario típico de reintento por timeout de red o reinicio de consumidor. Segundo, un índice único sobre EventId en el almacén de auditoría, que actúa como red de seguridad definitiva ante cualquier reentrega fuera de esa ventana (ej. un mensaje reprocesado desde una dead-letter queue horas después): el INSERT falla por violación de constraint y el evento se descarta sin generar una segunda fila. Sin esta segunda barrera, un log de auditoría con filas duplicadas perdería valor como evidencia.

El almacén de auditoría se implementa en dos niveles, no en una sola tecnología. Azure SQL en modo append-only cubre la consulta operativa de eventos recientes, para investigaciones y reportes del día a día. Ese mismo log se archiva periódicamente en Azure Data Lake en modo WORM, con retención regulatoria mínima de 7 años. Ese esquema está alineado con el estándar típico de la industria bancaria para trazabilidad AML (ver diagrama de contenedores, sección 6).

Al ser un consumidor asíncrono desacoplado (patrón Publish/Subscribe), un fallo o lentitud del servicio de auditoría nunca bloquea la transacción de negocio original. Se prioriza la disponibilidad del flujo transaccional sobre la escritura inmediata del log, consistente con el requisito de tolerancia a fallos.

El almacenamiento append-only (sin UPDATE/DELETE habilitados a nivel de permisos) es lo que hace al registro válido como evidencia forense/regulatoria. Ningún actor, incluido un administrador, puede alterar retroactivamente el historial. Eso cumple el principio de no repudio exigido por la normativa AML/KYC de la sección 3.

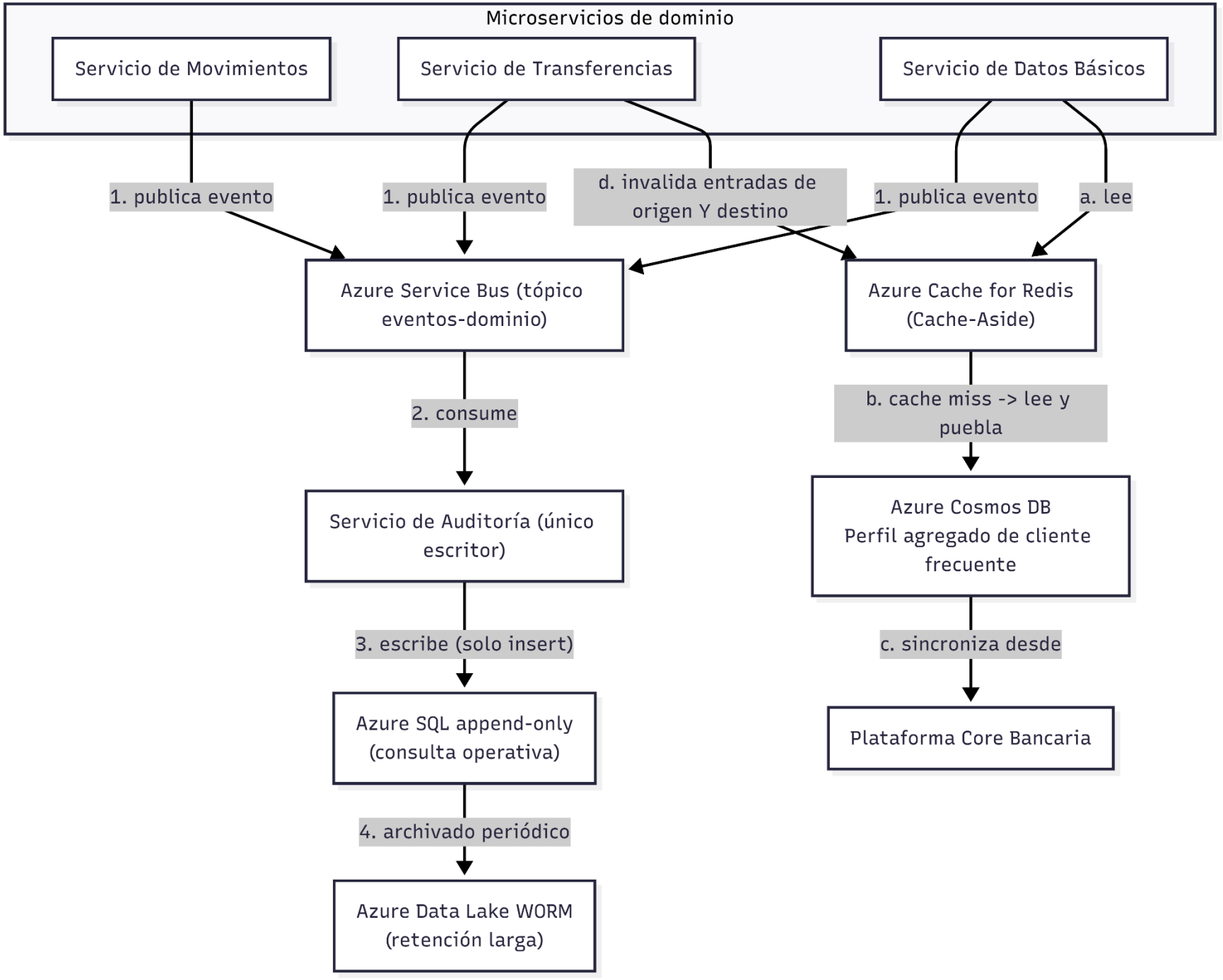
11.2 Persistencia para clientes frecuentes

Proponemos el patrón Cache-Aside (Lazy Loading) con Azure Cache for Redis, delante de la fuente de verdad (Core Bancario / Cosmos DB). Ahí almacenamos el perfil agregado de clientes de alta frecuencia de uso: datos básicos, últimos movimientos, cuentas/beneficiarios favoritos.

El acceso a datos bancarios sigue una distribución altamente sesgada: un subconjunto de clientes consulta con mucha más frecuencia que el resto. Cache-Aside evita cargar en caché datos que no se van a reutilizar, optimizando costo de memoria frente a beneficio real de latencia.

Al ser el propio servicio quien decide cuándo poblar/invalidar la caché (en vez de que la base de datos la empuje), se mantiene el control de consistencia. En una transferencia se invalidan explícitamente las entradas en caché de ambos clientes involucrados, origen y destino (cuando el destino también es cliente de BP), antes de confirmar la operación. Así evitamos que cualquiera de los dos vea un saldo desactualizado. Invalidar solo el lado origen dejaría al destinatario viendo su saldo viejo hasta que expire el TTL de la caché.

Persistencia para clientes frecuentes



12- Capa de integración: API Gateway y servicios

Todo acceso de los canales (SPA, móvil) a los sistemas backend pasa por Azure API Management como API Gateway único. Ahí se aplica autenticación (validación del JWT OAuth2.0), rate limiting, versionado de contratos y observabilidad centralizada, antes de enrutar al microservicio correspondiente.

12.1 Servicios base

Servicio	Responsabilidad	Sistemas que consume
Consulta de datos básicos	Perfil, productos y datos generales del cliente	Plataforma Core, Sistema Complementario de Cliente, caché de clientes frecuentes
Consulta de movimientos	Histórico de transacciones	Plataforma Core, caché de clientes frecuentes
Transferencias	Transferencias propias, intra BP e interbancarias, con protección de idempotencia	Plataforma Core (siempre); Servicio de Riesgo (intra-BP a terceros e interbancarias; **no** en propias, que solo llevan validación ligera de límites); Red Interbancaria (**solo** si el tipo es interbancaria, de forma asíncrona; ver diagrama de secuencia 8.3 para que pueda referenciarse)

12.2 Servicios adicionales propuestos

Se incorporan los siguientes servicios adicionales para mejorar rendimiento, resiliencia y cohesión de responsabilidades, más allá de los 3 iniciales:

Servicio adicional	Por qué se agrega
Riesgo / Antifraude	Aísla las reglas de scoring de fraude (que cambian con frecuencia y requieren iteración independiente de auditores/riesgo) del servicio de Transferencias, evitando que cada ajuste de regla de negocio de riesgo obligue a redesplegar la lógica transaccional.
AML / Transaction Monitoring	Disciplina distinta del antifraude en tiempo real: analiza patrones de estructuracion, umbrales regulatorios y cribado contra listas de sancion OFAC y ONU sobre el mismo flujo de eventos de auditoria de la seccion 11.1, como consumidor adicional, no como parte del scoring por transaccion del servicio de Riesgo.
Onboarding	Flujo con integraciones propias (IDV, KYC, IdP) y ciclo de vida distinto al resto: se beneficia de escalar y desplegar de forma independiente (picos por campañas de adquisición de clientes).
Auditoría	Único escritor del almacén inmutable (sección 11.1); aislarlo simplifica el control de permisos de escritura a nivel de infraestructura.
Orquestador de Notificaciones	Desacopla "qué pasó" (evento de dominio) de "cómo se notifica" (canal, plantilla, idioma); ver sección 13.
BFF Web y BFF Móvil (Backend for Frontend)	Ver justificación específica abajo.

***Patrón Backend for Frontend**: la SPA y la app móvil no piden lo mismo, la móvil cuida ancho de banda y batería, la web puede darse el lujo de pedir vistas más pesadas. Por eso usamos un BFF por canal, cada uno agrega y transforma las respuestas de los microservicios de dominio a la forma exacta que su front necesita. Así evitamos overfetching en móvil sin castigar a la SPA.*

Permite que los equipos de frontend evolucionen sus contratos de API sin coordinar cambios directamente sobre los microservicios de dominio, que sirven a ambos canales. Así se reduce el acoplamiento entre la cadencia de release de la app móvil y la del backend transaccional.

El BFF Web es stateful por diseño: retiene la sesión del usuario, y eso obliga a decidir explícitamente cómo escala. Descartamos sticky sessions, atan al usuario a una réplica específica del BFF, y eso choca de frente con el autoscaling y los rolling deployments de AKS, si esa réplica se cae, la sesión se cierra de golpe. Por eso la sesión vive en un almacén distribuido (Azure Cache for Redis) y no en memoria del pod. Con la sesión en Redis, cualquier réplica del BFF atiende cualquier request, y el Horizontal Pod Autoscaler puede escalar hacia arriba o abajo sin tocar las sesiones activas.

12.3 Gestión de contratos y versionado de APIs

Diseñamos toda API expuesta mediante Azure API Management bajo el enfoque contract first. Usamos OpenAPI 3.x como especificación única de la interfaz: el contrato se define y revisa antes de implementar el servicio. De ahí salen tanto la documentación pública (portal de desarrolladores de APIM) como los clientes y stubs de prueba. Esto evita el clásico "la documentación se desincroniza del código" y le da al equipo de front-end un contrato estable contra el cual trabajar en paralelo al backend.

El versionado se maneja por segmento de URL (v1, v2) y no por header, por dos razones: queda explícito en logs y trazas (más fácil auditar qué versión consume cada canal), y APIM aplica políticas de forma granular por versión sin ambigüedad. La política de deprecación mantiene hasta N-2 versiones activas al mismo tiempo, con un ciclo de vida documentado. Al liberar v3, anunciamos la deprecación de v1 con seis meses de anticipación. Eso incluye el header estándar Sunset (RFC 8594) en cada respuesta de la versión deprecada, para que los clientes puedan reaccionar de forma automatizada.

Un cambio *breaking* nunca se introduce en una versión existente; siempre implica una nueva versión mayor.

13- Notificaciones

La normativa exige notificar al cliente cada movimiento realizado. Se implementan 2 canales orquestados por el Servicio Orquestador de Notificaciones, que consume los eventos de dominio (TransferenciaRealizada, ClienteRegistrado, etc.) publicados en Azure Service Bus:

Canal	Tecnología	Rol
Push	Azure Notification Hubs	Canal primario, inmediato, de menor costo por envío; requiere app instalada y con permisos activos.
SMS / Email	Proveedor externo especializado (ej. Twilio para SMS, SendGrid para email)	Canal de respaldo garantizado, no depende de que la app esté instalada/conectada.

Redundancia de canal por disponibilidad: si el dispositivo está sin conexión, el usuario desinstaló la app o revocó permisos de notificación, el push simplemente no llega. SMS/Email opera sobre un canal independiente (red celular / correo). Eso permite cumplir la obligación normativa de notificar aun en ese escenario.

Usamos proveedores externos especializados para SMS/Email en vez de construir la integración directa con operadores. Así delegamos el cumplimiento de entregabilidad, reputación de remitente y regulaciones locales de mensajería (por ejemplo, el registro de remitentes SMS) a un tercero cuyo negocio es justamente garantizar esa entrega, y reducimos el riesgo operativo de BP.

El orquestador aplica reglas simples (tipo de evento → plantilla + canal(es) requeridos), reintentos con backoff exponencial por canal, y publica el resultado de envío también hacia auditoría (todo intento de notificación queda trazado).

Igual que en Auditoría (sección 11.1), el orquestador deduplica por EventId antes de enviar. Sin esto, un evento reentregado por Service Bus podría traducirse en el mismo cliente recibiendo dos veces el SMS o push de "se realizó tu transferencia" o "transferencia exitosa".

14- Alta disponibilidad, tolerancia a fallos, DR, seguridad, monitoreo y auto-healing

Pilar	Mecanismo en Azure	Detalle
Alta disponibilidad	AKS multi-zona · Azure SQL zoneredundant · APIM Premium multi-región · Front Door	Los nodos de AKS se distribuyen en al menos 3 zonas de disponibilidad de la región primaria; Azure SQL usa configuración zone-redundant; Front Door enruta automáticamente al backend/región saludable más cercana.
Tolerancia a fallos	Circuit Breaker + Retry con backoff exponencial (adaptadores, sección 7) · colas muertas (dead-letter) en Service Bus · Saga con compensación	Ningún fallo de un sistema externo (Core, banco destino, riesgo) debe propagarse como caída total; se degrada de forma controlada y se compensa cuando aplica. Modo degradado orientado al cliente: si la Plataforma Core no responde, Datos Básicos y Movimientos no muestran una pantalla en blanco, siguen sirviendo desde la caché Cache-Aside (Redis, sección 11.2) con un aviso visible de que la información puede no estar actualizada. Transferencias, que sí depende de una confirmación de saldo en tiempo real, no tiene ese margen (sección 7) y rechaza la operación de forma explícita en vez de aproximarla.
Recuperación ante desastres (DR)	Región secundaria pareada de Azure · Azure SQL auto failover groups · Storage con replicación geo-redundante · Service Bus georecuperación	Estrategia *activo/pasivo* con objetivo **RTO ≤ 60 min** y **RPO ≤ 5 min** para datos transaccionales (razonable para banca digital sin llegar al costo de activo/activo global); failover de base de datos automático, failover de aplicación orquestado vía Front Door. ***El RPO ≤ 5 min*** es lo que entregan de forma nativa y sin trabajo adicional los mecanismos de replicación geo-redundante ya elegidos (Azure SQL auto failover groups, RA-GRS). Pedir un RPO menor, por ejemplo 0 con activo/activo síncrono, obligaría a replicación síncrona entre regiones, lo que añade latencia a cada escritura y va en contra del requisito de baja latencia; pedir uno mayor arriesgaría perder transacciones de clientes ya confirmadas, algo que un banco no puede aceptar. ***El RTO ≤ 60 min***, por su parte, es el tiempo típico de un failover *orquestado* (no automático a nivel de segundos) que incluye verificación de integridad antes de aceptar tráfico de nuevo: ese margen se prefiere frente a un failover completamente automático e instantáneo, que podría promover una región secundaria con datos aún no del todo consistentes.
Seguridad	Azure Key Vault · Managed Identities · WAF en Front Door · Private Endpoints · Microsoft Defender for Cloud · Azure Policy	Sin secretos embebidos (Managed Identity + Key Vault); bases de datos y Service Bus sin exposición pública (Private Endpoints dentro de la VNet); cifrado TDE en reposo y TLS 1.2+ en tránsito; políticas de gobierno que impiden desplegar recursos fuera de estándar (ej. storage público).
Monitoreo	Azure Monitor + Application Insights · Log Analytics Workspace · Azure Service Health	Trazabilidad distribuida extremo a extremo (correlation ID por request, propagado desde APIM hasta cada microservicio); alertas basadas en SLI/SLO (latencia p95, tasa de error); monitoreo sintético sobre endpoints críticos (login, transferencia).

Baja latencia	Azure Front Door (enrutamiento al PoP de borde más cercano) · Azure CDN para assets estáticos de la SPA · Región primaria elegida por proximidad geográfica a la base de usuarios · Cache-Aside (sección 11.2) resolviendo el camino caliente sin ir al Core	Objetivo declarado: **p95 < 300 ms** para el "camino caliente" (consulta de datos básicos/movimientos, la interacción más frecuente), servido en su mayoría desde Redis en la misma región, no desde la Plataforma Core. • El mismo Azure Front Door que da alta disponibilidad (enruta a la región/backend saludable) resuelve baja latencia como efecto del mismo mecanismo: enruta al punto de presencia de borde más cercano al usuario antes de tocar el backend, sin necesitar un componente adicional solo para esto. • Elegir la región Azure primaria por proximidad a la base de usuarios de BP (en vez de por costo mínimo del cómputo) es la decisión de mayor impacto en latencia real: ningún ajuste fino de caché o CDN compensa una región primaria mal ubicada geográficamente frente a dónde están los clientes.
Excelencia operativa	Infraestructura como código (Bicep/Terraform) · CI/CD con despliegues blue/green o canary · runbooks de incident response · **chaos engineering** (Azure Chaos Studio)	Todo cambio de infraestructura es versionado y reproducible; los despliegues a producción no son "big bang", minimizando el impacto de un release defectuoso. Además, la resiliencia diseñada en la sección 7 (Circuit Breaker, Saga, colas muertas) se **valida activamente** con experimentos de fallo controlado que no es productivo (caída simulada del Core, latencia inyectada al banco destino) en vez de asumir que el diseño en papel funciona como se espera. El estado del clúster AKS se reconcilia de forma continua vía GitOps (Flux o Argo CD): el estado deseado vive versionado en el repositorio, y un controlador dentro del clúster aplica cualquier divergencia automáticamente, en vez de depender de que cada cambio se empuje manualmente vía pipeline. Esto también facilita el rollback (revertir el commit revierte el clúster) y deja una auditoría exacta de quién cambió qué en producción.
Auto healing	Liveness/readiness probes en AKS · reinicio automático de pods · node autorepair de AKS · Horizontal Pod Autoscaler	Un microservicio que deja de responder a su probe de salud es reiniciado automáticamente por el orquestador sin intervención humana; los nodos AKS con fallas de hardware/SO se reemplazan de forma automática.

DevSecOps y seguridad de la cadena de suministro

La resiliencia de la aplicación en runtime no es suficiente sin controles sobre cómo llegó el código y las imágenes a producción. Por eso el pipeline de CI/CD incorpora, como *gates* obligatorios antes del despliegue:

- **SAST** (Análisis estático) con GitHub Advanced Security o SonarQube sobre cada pull request, bloqueando merge ante vulnerabilidades de severidad alta.
- **Escaneo de dependencias** (SCA) con Dependabot / Snyk para detectar librerías con CVEs conocidos antes de que lleguen a un contenedor.
- **Escaneo de imágenes de contenedor** con Microsoft Defender for Containers / Trivy en el registry (Azure Container Registry), impidiendo el pull a producción de imágenes con vulnerabilidades críticas no remediadas.

- **Firmamos las imágenes con Notary v2 / Cosign y verificamos esa firma en el clúster AKS mediante un admission controller (Kyverno o similar). Un pod solo se admite si su imagen viene firmada por el pipeline oficial. Eso cierra la posibilidad de que una imagen manipulada o de un origen no confiable llegue a ejecutarse.**
- **Generamos un SBOM (Software Bill of Materials)** por build, en formato CycloneDX, y lo archivamos como evidencia auditable. Así, ante un CVE nuevo (otro incidente tipo Log4Shell, por ejemplo) podemos responder en minutos qué servicios están expuestos, en vez de auditar el estado de cada repositorio a mano.

Este conjunto de controles reduce la superficie de la cadena de suministro (*supply chain attack surface*), un vector que la industria financiera ya trata como requisito.

Scrubbing de PII en telemetría: por defecto, Application Insights captura el cuerpo completo de requests/responses y los parámetros de query, y ahí pueden colarse números de cuenta, documentos de identidad o datos biométricos si no filtramos antes de enviar nada. Para eso configuramos un Telemetry Processor en el SDK de OpenTelemetry que enmascara o elimina los campos clasificados como Restringido o Confidencial (sección 3) antes de que la traza salga del proceso. No es una limpieza posterior sobre datos ya guardados, es un filtro en el origen.

Observabilidad basada en estándares abiertos: instrumentamos los microservicios vía OpenTelemetry (traces, metrics, logs) en lugar de un SDK propietario de Azure. La telemetría se envía hoy a Application Insights como backend, pero el instrumental no queda atado a él.

Custodia de claves criptográficas.

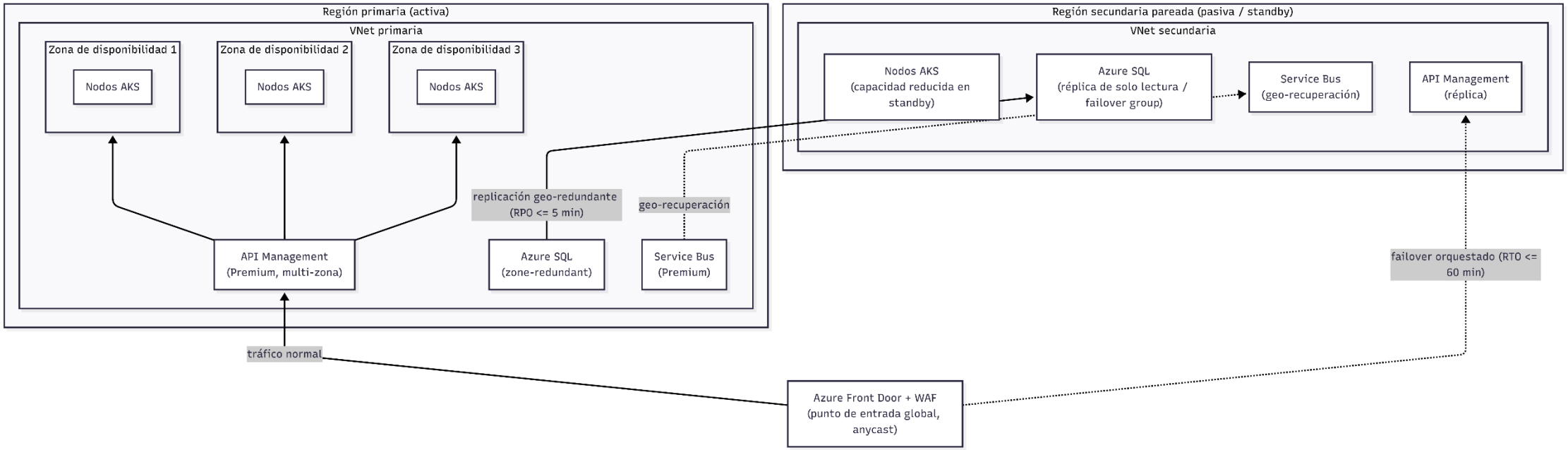
Para los datos con mayor sensibilidad regulatoria (información financiera y de auditoría), el cifrado en reposo se configura bajo el modelo **BYOK (Bring Your Own Key) / Customer Managed Keys**, con las claves maestras custodiadas en **Azure Key Vault Managed HSM**. Se prefiere sobre las claves gestionadas por la plataforma (*platform managed keys*) por dos razones.

- Primero, la separación de custodia: BP retiene control criptográfico sobre sus datos incluso frente al proveedor de nube. Es un requisito que varios reguladores financieros exigen o recomiendan explícitamente al operar en jurisdicciones con leyes de protección de datos estrictas.
- Segundo, la posibilidad de rotar claves y revocarlas ante un incidente, algo que, con claves gestionadas por la plataforma, el cliente no controla. Aplica a Azure SQL (TDE con CMK), Storage Accounts (incluyendo el Data Lake de auditoría) y Cosmos DB.

Justificación de la estrategia HA/DR elegida

AKS con auto-healing y multi-zona, frente a VMs administradas manualmente: Kubernetes reconcilia continuamente el estado deseado (número de réplicas, salud de pods). Eso traduce directamente el requisito de auto-healing en un mecanismo nativo de la plataforma, en vez de scripts de monitoreo ad-hoc. El resultado es un tiempo medio de recuperación (MTTR) menor.

DR activo/pasivo con RTO/RPO acotados, en vez de activo-activo global. Para el alcance de este ejercicio, un esquema activo/activo multi-región duplicaría el costo de cómputo y sumaría complejidad de resolución de conflictos de escritura, por ejemplo dos transferencias concurrentes desde regiones distintas sobre la misma cuenta. Un esquema activo/pasivo con failover orquestado cumple los objetivos de continuidad de negocio típicos de banca digital regional, con un costo y una complejidad operativa razonables.



15- Infraestructura Azure y consideraciones de costos

Componente	Servicio Azure	Driver principal de costo
Cómputo de microservicios	Azure Kubernetes Service (AKS)	Número y tamaño de nodos (VMs), con autoscaling: el costo sigue la demanda real en vez de una capacidad fija sobredimensionada.
API Gateway	Azure API Management (tier Premium)	Se requiere Premium para soporte multi-región y VNet integration; es el tier más costoso pero necesario dado el requisito de HA/DR.
Perímetro / WAF	Azure Front Door + WAF	Tráfico procesado (GB) y reglas WAF activas.

Datos transaccionales	Azure SQL (por servicio, zone redundant)	vCores/DTUs reservados; se dimensiona por servicio según su carga real (Movimientos y Transferencias requieren más que Onboarding).
Perfil de clientes frecuentes	Azure Cosmos DB + Azure Cache for Redis	Cosmos: unidades de request (RU/s), se recomienda modo *serverless* si el tráfico es variable; Redis: tamaño de caché según % de clientes frecuentes a cubrir.
Mensajería	Azure Service Bus (Premium para aislamiento y mayor throughput)	Unidades de mensajería reservadas.
Identidad	Azure AD B2C	Facturación por usuarios activos mensuales (MAU), costo directamente ligado a la adopción y no a infraestructura fija.
Onboarding/IDV	Proveedor especializado (Jumio/FaceTec/Onfido) + Azure AI Vision (verificaciones secundarias)	Facturación por verificación realizada (costo variable por nuevo cliente onboarded).
Notificaciones	Azure Notification Hubs + proveedor SMS/Email externo	Notification Hubs: por push enviado; SMS/Email: por mensaje, típicamente el costo variable más sensible al volumen.
Observabilidad	Azure Monitor / Application Insights / Log Analytics	Volumen de telemetría ingerida (GB/día), controlado con políticas de retención y muestreo (sampling) en trazas de alto volumen.
Secretos	Azure Key Vault	Costo marginal (por operación), no es un driver relevante del total.

Palancas de optimización de costo

- **Autoscaling en todo el stack de cómputo** (AKS, APIM Premium con unidades escalables) para pagar por demanda real, con picos esperables en horarios bancarios y campañas de onboarding.
- **Reserva de capacidad (Reserved Instances / Savings Plan)** para la carga base de nodos AKS que sabemos constante, combinada con autoscaling solo para el excedente.
- **Modo serverless en Cosmos DB** si el patrón de acceso de "clientes frecuentes" resulta más intermitente de lo esperado, evitando pagar RU/s provisionados ociosos.
- **Muestreo de telemetría** en Application Insights para transacciones de bajo riesgo, reservando trazabilidad al 100% para flujos críticos (autenticación, transferencias).

Orden de magnitud de referencia, no es cotización formal: supone ~200.000 clientes activos mensuales, tráfico típico de banca digital regional, un solo ambiente productivo en HA. La infraestructura descrita se ubica en torno a USD 22.000/mes en régimen. El rango razonable va de USD 20.000 a 25.000/mes, según la variabilidad de usuarios activos mensuales (Azure AD B2C) y el volumen de notificaciones push/SMS, los dos componentes con el driver de costo más elástico. Esto no incluye el costo variable del proveedor IDV, que factura por verificación y crece con la tasa de onboarding, no con el tiempo. Los mayores drivers dentro de ese rango son:

- AKS + API Management Premium (~40-50% del total, por ser los únicos componentes dimensionados para HA multi-zona todo el tiempo) y observabilidad + mensajería a escala (~15-20%)
- el resto (SQL, Redis, Cosmos, Key Vault, notificaciones) es comparativamente marginal salvo picos de volumen de SMS/push.

Este rango es una referencia de orden de magnitud, no una cotización. Ya que el ejercicio no exige un cálculo exacto, sino evidenciar manejo consciente de costos en las decisiones de arquitectura, como el uso de autoscaling y tiers dimensionados por servicio en lugar de aprovisionar de forma uniforme.

16- Arquitectura Front-End: SPA y Móvil

16.1 SPA Web

Se recomienda **Angular** como framework de la SPA, evaluado frente a React.

Angular es un framework integral y opinionado: trae routing, formularios reactivos, inyección de dependencias y cliente HTTP incluidos oficialmente. Eso reduce la variabilidad de decisiones técnicas entre equipos, algo que en un contexto bancario pesa mucho, porque ahí la consistencia y auditabilidad del código valen más que la flexibilidad. React es distinto: al ser solo una librería de vista, obliga a ensamblar y mantener alineado un conjunto mayor de librerías de terceros.

TypeScript es ciudadano de primera clase en Angular, no una opción añadida. Su inyección de dependencias facilita aislar y mockear servicios de integración con el API Gateway en pruebas unitarias. Eso es relevante para una app que maneja datos financieros, donde la cobertura de pruebas es un criterio de calidad esperado.

Se descarta Vue por menor adopción/soporte empresarial en el ecosistema bancario típico y por un ecosistema de librerías de terceros menos maduro que React/Angular para necesidades como manejo de formularios financieros complejos.

Manejo de estado

Recomendamos NgRx (Redux adaptado a Angular sobre RxJS) para el estado transaccional compartido: sesión, saldos en pantalla, resultado de operaciones en curso. Reservamos el estado local de componente para lo puramente de UI, como validación de formularios o toggles.

Un store unidireccional con acciones explícitas deja un registro reproducible de "qué cambió el estado y cuándo" en el cliente. Eso sirve para depurar incidentes reportados por usuarios sobre saldos o transferencias, sin tener que reconstruir la secuencia de eventos a ciegas.

RxJS ya viene en Angular y encaja naturalmente con flujos asíncronos, como el refresco de saldo tras una transferencia o la actualización en tiempo real de notificaciones. Evita mezclar callbacks y promesas de forma inconsistente entre módulos.

Accesibilidad

Exigimos cumplimiento de WCAG 2.1 nivel AA, con Angular CDK a11y como base: manejo de foco, roles ARIA, navegación por teclado. Esto es consistente con el estándar de accesibilidad que reguladores financieros de varios países ya exigen a la banca digital.

Disponibilidad offline / PWA

La SPA se empaqueta como **PWA con Service Worker** únicamente para contenido de solo lectura ya cacheado (ej. último histórico de movimientos consultado), nunca para operaciones de escritura (transferencias) mientras no haya conectividad. Una transferencia iniciada sin conexión y "encolada" para reintentar automáticamente es un riesgo de doble ejecución que no compensa la mejora de experiencia, por lo que se descarta explícitamente ese caso de uso.

16.2 Aplicación móvil multiplataforma

Se evaluaron dos frameworks multiplataforma, como exige el ejercicio: **React Native** y **Kotlin Multiplatform Mobile (KMP)**. Se recomienda **KMP**.

Opción	Naturaleza
React Native	UI multiplataforma con JavaScript/TypeScript sobre un puente (bridge) o JSI hacia componentes nativos.
Kotlin Multiplatform Mobile	Comparte solo la lógica (dominio, red, persistencia); la UI se implementa nativa en cada plataforma (Jetpack Compose en Android, SwiftUI en iOS).

Seguridad y control nativo: el onboarding con reconocimiento facial y la biometría de acceso (secciones 9-10) requieren integración profunda con las APIs de seguridad de cada plataforma (Android BiometricPrompt/Keystore, iOS Face ID/Secure Enclave, SDKs nativos de los proveedores IDV). KMP nos deja implementar esas integraciones de forma nativa, con acceso directo a las APIs de la plataforma. React Native, en cambio, suma una capa de puente que mete fricción y que en el pasado se ha señalado como superficie de riesgo extra para SDKs sensibles a la plataforma, como biometría o criptografía de hardware.

Rendimiento: KMP compila la lógica compartida a código nativo, vía Kotlin/Native para iOS y JVM/Android directamente, y deja la UI 100% nativa. Así evita el overhead del puente de renderizado que históricamente ha sido el cuello de botella de rendimiento de React Native en listas grandes (historial de movimientos) y animaciones complejas.

Descartamos Flutter, la tercera alternativa evaluada, porque requiere su propio motor de renderizado (Skia). Ese motor no reutiliza los componentes de accesibilidad nativos de cada plataforma de forma tan directa como KMP+Compose/SwiftUI, algo relevante para cumplir estándares de accesibilidad bancaria.

17- Conclusiones

La arquitectura propuesta resuelve los requerimientos funcionales del ejercicio: consulta los datos, movimientos, transferencias propias e interbancarias, onboarding biométrico y notificaciones. Lo hace con una arquitectura de microservicios desacoplada, expuesta mediante un API Gateway único.

OAuth 2.0/OIDC queda como estándar de autenticación, configurado, no construido, sobre el producto de identidad que ya existe en BP.

Las decisiones de mayor peso (Azure como nube, microservicios sobre AKS, Saga para transferencias, Cache-Aside para clientes frecuentes, log de auditoría inmutable basado en eventos, KMP para el canal móvil) tienen al menos dos argumentos teóricos cada una. Todas se contrastaron contra alternativas descartadas. Priorizamos siempre los atributos de calidad no negociables en banca digital: seguridad, disponibilidad, auditabilidad y cumplimiento regulatorio. Sin renunciar a que el sistema pueda evolucionar en el futuro de forma independiente por dominio.